



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G5

System Integration and Architecture

System Design Document (SDD)

Project Title: CheckMate Academy

Prepared By: Paradela, Lawrence Anthony D.

Version: 1

Date: 2/20/2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/20/2026	Lawrence Anthony D. Paradela	Initial draft	Draft
0.2	2/27/2026	Lawrence Anthony D. Paradela	Added API specifications	Review
1.0	4/25/2026	Lawrence Anthony D. Paradela	Baseline version for development	Review
1.1	5/10/2026	Lawrence Anthony D. Paradela	Overhauled architecture.	Review
1.2	5/25/2026	Lawrence Anthony D. Paradela	Aligned SDD specifications with final web, mobile, and database implementation.	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

CheckMate Academy is a specialized platform designed to bridge the gap between chess students and professional coaches. Built using a decoupled architecture, React for the web presentation layer, a stateless Spring Boot (Java) backend, and a PostgreSQL database hosted on Supabase, it enables real-time lesson scheduling, interactive tactical board sessions, and secure communication.

1.2 Objectives

1. Provide a seamless Real-time Scheduling system for chess lessons with automated conflict prevention.
2. Implement an enterprise-grade, stateless backend architecture using Spring Boot.
3. Establish a robust Trust & Safety layer, allowing Administrators to moderate users, verify ELO ratings, and resolve platform reports.
4. Deliver a synchronized "Live Session" environment where coaches and students can interact over a shared digital chessboard with integrated chat.

1.3 Scope

Included Features:

- Custom user registration and authentication (Role-based: Student, Coach, Admin) using BCrypt and stateless tokens.
- Coach directory with availability matrices, filtering, and conflict-aware booking.
- Live Lesson environment with automated FEN/PGN board state synchronization.
- Dedicated Admin Dashboard for account flagging and report resolution.
- Strict client-side session isolation (sessionStorage) to allow multi-role testing on the same machine.
- A native Android application built in Kotlin (XML layouts) that communicates dynamically via a REST API client (Retrofit) with the Spring Boot backend to

support lesson scheduling, profiles, completed lesson history, and live session chessboards.

Excluded Features:

- Real payment gateway integration.
- In-app video conferencing (will rely on external links like Google Meet).
- iOS applications (focusing on responsive web MVP).
- Advanced analytics dashboard.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the CheckMate Academy system. It provides detailed requirements, architectural decisions, API contracts, database design, and an implementation roadmap to guide development and ensure seamless integration between the React frontend and Supabase backend.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: CheckMate Academy

Domain: Education / Scheduling Platform

Primary Users:

1. Students (Chess learners)
2. Coaches (Experienced players/instructors)
3. Administrators (Platform moderators and security officers)

Problem Statement: Students struggle to find and schedule dedicated time with qualified chess coaches without heavy administrative back-and-forth.

Solution: A centralized platform focusing on real-time availability, direct booking, and clear tracking of a student's chess progress.

2.2 Core User Journeys

Journey 1: Student Books a Lesson

1. User registers/logs in as a "Student".
2. Browses the directory of available coaches, filtering by playstyle or date.
3. Selects an open time slot; system verifies no overlapping conflicts exist.
4. Booking is confirmed as "PENDING" and awaits coach approval.

Journey 2: Coach Conducts a Session

1. Coach logs in and accepts a pending lesson request.
2. At the scheduled time, Coach enters the "Live Session" view.
3. Coach moves pieces on the tactical board; the system synchronizes the FEN and PGN history to the Student's screen.
4. Coach and Student utilize the live chat (saved as JSON notes) to communicate.
5. Coach marks the lesson as "COMPLETED", archiving the board state for future review.

Journey 3: Admin Moderation

1. Administrator logs into the secure Admin Panel.
2. Reviews incoming Trust & Safety reports submitted by users during live sessions.
3. Flags malicious accounts or explicitly verifies the ELO ratings of Coach profiles.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (Register, login, logout via Supabase Auth).
2. Coach directory (List, detail, search).
3. Real-time booking system (Select slot, confirm, update).
4. User dashboards (View upcoming lessons).

SHOULD HAVE

1. ELO rating tracking on user profiles.
2. Basic input validation feedback.
3. Responsive design for all screen sizes.

COULD HAVE

1. Lesson notes or tactical puzzle assignments (Live Session Board).
2. Real-time notifications for new bookings.
3. Admin Trust & Safety Dashboard for moderation.

WON'T HAVE

1. Real payment processing.
2. Multi-language support.

2.4 Detailed Feature Specifications

Feature: Authentication & Identity Management

- **Screens:** Registration, Login.
- **Fields:** Email, Password, Role, Chess Username, ELO.
- **Security:** Handled by Spring Boot SecurityFilterChain. Passwords hashed via BCryptPasswordEncoder. Sessions managed client-side via sessionStorage to prevent cross-tab contamination.
- **API Endpoints:** POST /api/auth/register, POST /api/auth/login

Feature: Coach Availability & Directory

- **Screens:** Directory Listing, Coach Profile Settings.
- **Functions:** Coaches set weekly availability blocks. System automatically calculates overlapping conflicts and prevents double-booking.
- **API Endpoints:** GET /api/users/coaches, GET /api/users/coaches/available-slots, PUT /api/users/coaches/{id}/profile

Feature: Live Session & Tactical Board

- **Screens:** Active Lesson View.
- **Functions:** Integrates chess.js and react-chessboard. Utilizes a lightweight 2000ms short-polling mechanism to synchronize board state (FEN), move history (PGN), and chat messages between Coach and Student without needing WebSockets.
- **API Endpoints:** PUT /api/lessons/{id}/board, PUT /api/lessons/{id}/notes

Feature: Trust & Safety (Admin)

- **Screens:** Admin Dashboard.
- **Functions:** View all platform users, update RBAC roles, toggle account flags, verify ELOs, and resolve user-submitted behavior reports.

- **API Endpoints:** GET /api/admin/users, PUT /api/admin/users/{id}/flag, POST /api/reports

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new user
- When I enter a valid email, strong password, and select the "Student" role
- And click "Register"
- Then my account should be securely created in the PostgreSQL database with a BCrypt hashed password
- And I should be redirected to the Login page

AC-2: Coach Availability Management

- Given I am logged in as a Coach
- When I navigate to my schedule settings
- And I add a specific day and time range (e.g., Monday 2:00 PM - 5:00 PM)
- And I click "Save Complete Profile"
- Then the system must validate that no time slots overlap on the same day
- And the time slots should be saved to the database to reflect on the public directory

AC-3: Lesson Booking Flow

- Given I am logged in as a Student
- When I view the Booking page
- And I select an available time slot
- And I click "Confirm Booking"
- Then a new record should be created in the lessons table with a "PENDING" status
- And the selected time slot should instantly register as a conflict for any other students attempting to book that exact window

AC-4: Coach Lesson Approval

- Given I am logged in as a Coach
- When I view my Coach Dashboard
- And I see a new lesson request with a "PENDING" status
- And I click "Accept"
- Then the lesson status should update to "ACCEPTED" in the database
- And the "Join Lesson" button should become available for the Live Session

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- **API Response Time:** API backend queries complete within 500ms for scheduling conflicts and profile lookups. Cold starts on Render cloud hosting may take up to 60 seconds.
- **Web Page Load Time:** ≤ 3 seconds on broadband connections.
- **Live Synchronization:** Board state and chat updates via short-polling must reflect on connected clients within 2.5 seconds.
- **Concurrency:** Support up to 100 concurrent users interacting with the platform simultaneously.

3.2 Security Architecture & Authentication Flow

Given the platform's emphasis on information security, CheckMateAcademy employs a decoupled, stateless security architecture:

- **Stateless Authentication Mechanism:** When a user logs in, the Spring Boot backend verifies the BCrypt-hashed password and generates a unique session token. The server immediately discards the transaction memory, remaining entirely stateless.
- **Client-Side Session Isolation:** The frontend strictly utilizes sessionStorage. Tokens are bound exclusively to the active browser tab, preventing cross-tab data contamination and ensuring that closing the tab instantly destroys the authorization token.
- **Role-Based Access Control (RBAC):**
 - * *Frontend:* The DashboardFactory acts as a gateway router, reading the user's role and dynamically routing them to isolated component trees.
 - *Backend:* The SecurityFilterChain permits access to the API endpoints (/api/auth/**, /api/users/**, /api/lessons/**, /api/admin/**, /api/reports/**) with CORS configured using environment variables. Access controls and session checks are verified on the client side (via sessionStorage tokens in React and memory extras in Android) to support dual web/mobile platform compatibility.
- **Data Privacy:** Environment variables (.env and application.properties) are utilized to keep Supabase PostgreSQL database credentials strictly out of the compiled source code.

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions).
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+).
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+, iOS 15+, Android 12+.

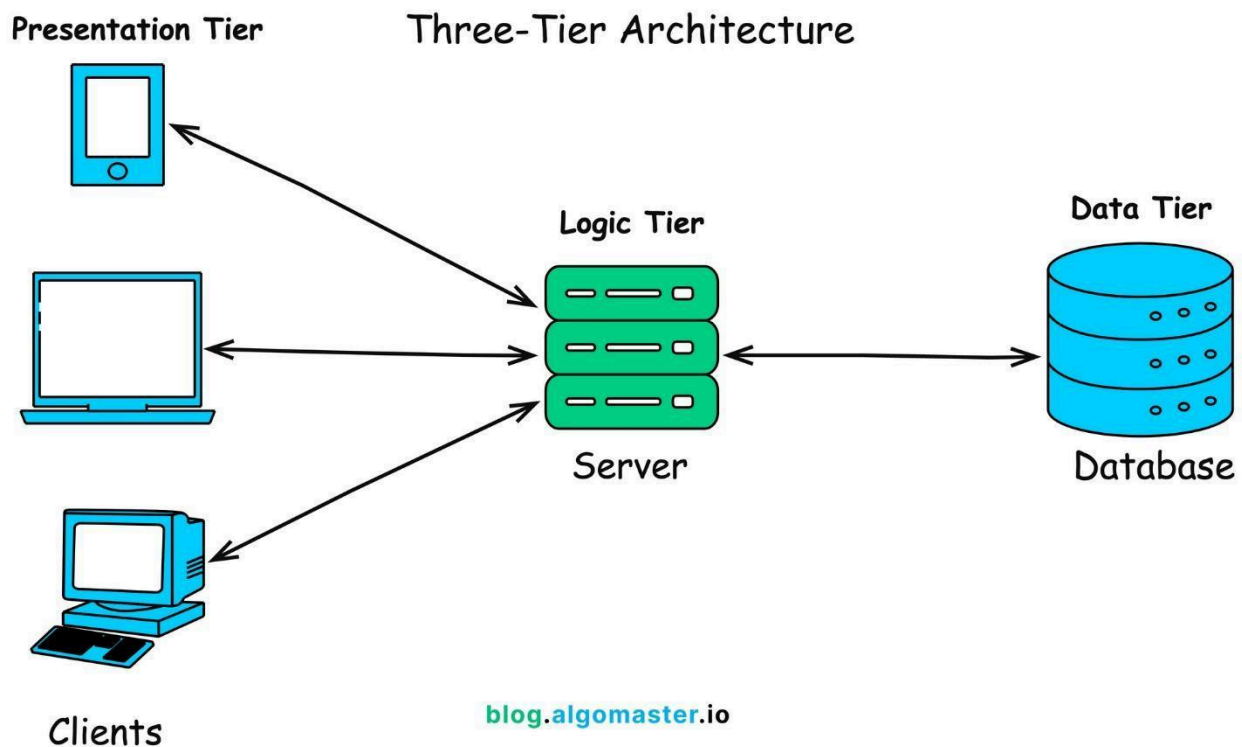
3.4 Usability Requirements

- New students must be able to complete their first lesson booking within 5 minutes of landing on the site.
- WCAG 2.1 Level AA compliance for web accessibility (proper contrast ratios).
- Consistent navigation across all isolated routing trees (Student Dashboard, Coach Dashboard, Admin Dashboard).
- Clear error messages with recovery options directly from the Spring Boot API error responses.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** PostgreSQL 14+

- **Web Frontend:** React 18, JavaScript(JS/JXS), Vanilla CSS, Axios
- **Mobile:** Kotlin, XML-Based, Retrofit, AndroidBridge(web-based chessboard)
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Render (Backend Web Service), Render (Web Static Site), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[backend-service-url]/api
Format	JSON for all requests/responses
Authentication	Bearer token (Custom Session UUID) in Authorization header
Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "<email>",</pre>

	<pre> "password": "<password>", "fullName": "<full_name>", "role": "<student_or_coach>", "chessUsername": "<username>", "currentElo": <integer> } </pre>
Response Structure	<pre> { "success": true, "message": "Login successful!", "token": "session-<uuid>", "role": "Student_or_Coach", "userId": "<uuid>", "data": { "user": { "id": "<uuid>", "email": "<email>", "fullName": "<full_name>", "role": "<student_or_coach>", "currentElo": 1200, "chessUsername": "<username>", "isFlagged": false, "eloVerified": false }, "accessToken": "session-<uuid>" } } </pre>

User Login

Description	Update Student/Coach profile info (ELO, name, and chess username).
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre> { "email": "<email>", "password": "<password>" } </pre>
Response Structure	<pre> { "success": boolean, </pre>

	<pre> "data": { "user": { "id": "<uuid>", "email": "<email>", "fullName": "<full_name>", "role": "<student_or_coach>", "currentElo": <integer> }, "accessToken": "session-<uuid>" } </pre>
--	--

Fetch Coach Availability

Description	Retrieves all open time slots for a specific coach.
API URL	/api/users/coaches/{coachId}/profile
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (Custom Session UUID)
Request Payload	
Response Structure	<pre> [{ "coachId": "<uuid>", "fullName": "<name>", "currentElo": 1500, "eloVerified": true, "bio": "Coach bio text...", "specialties": "Aggressive, Openings", "availabilities": [{ "dayOfWeek": 1, "startTime": "14:00:00", "endTime": "17:00:00" }] }] </pre>

Get Lessons Lists

Description	Fetch scheduled and completed lessons.
API URL	/api/lessons/coach/{coachId} (For Coaches) /api/lessons/student/{studentId} (For Students)
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (Custom Session UUID)
Request Payload	
Response Structure	[<pre> { "id": "<uuid>", "coachId": "<uuid>", "studentId": "<uuid>", "studentName": "<name>", "coachName": "<name>", "startTime": "2026-05-26T14:00:00", "endTime": "2026-05-26T15:00:00", "status": "ACCEPTED", "notes": "[]", "boardState": "start", "pgnHistory": "" }] </pre>

Admin Moderation & ELO Verification

Description	Get all user conduct reports or resolve them.
API URL	/api/reports (GET, retrieves list sorted newest first) /api/reports/{reportId}/resolve (PUT, sets status to "RESOLVED")
HTTP Method	GET/PUT
Format	
Authentication	
Request Payload	
Response Structure	

Behavior Reports

Description	Fetch scheduled and completed lessons.
API URL	/api/reports (GET, lists all reports newest first) /api/reports/{reportId}/resolve (PUT, sets status to "RESOLVED")
HTTP Method	GET/PUT
Format	JSON for all requests/responses
Authentication	Bearer token (Custom Session UUID)
Request Payload	
Response Structure	

5.3 Error Handling

HTTP Status Codes

- **200 OK - Successful request**
- **201 Created** - Resource created
- **400 Bad Request** - Invalid input
- **401 Unauthorized** - Authentication required/failed
- **403 Forbidden** - Insufficient permissions
- **404 Not Found** - Resource doesn't exist
- **409 Conflict** - Duplicate resource
- **500 Internal Server Error** - Server error

Error Code Examples

Example 1	<pre>json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2026-05-25T10:30:00Z" }</pre>
Example 2	<pre>json</pre>

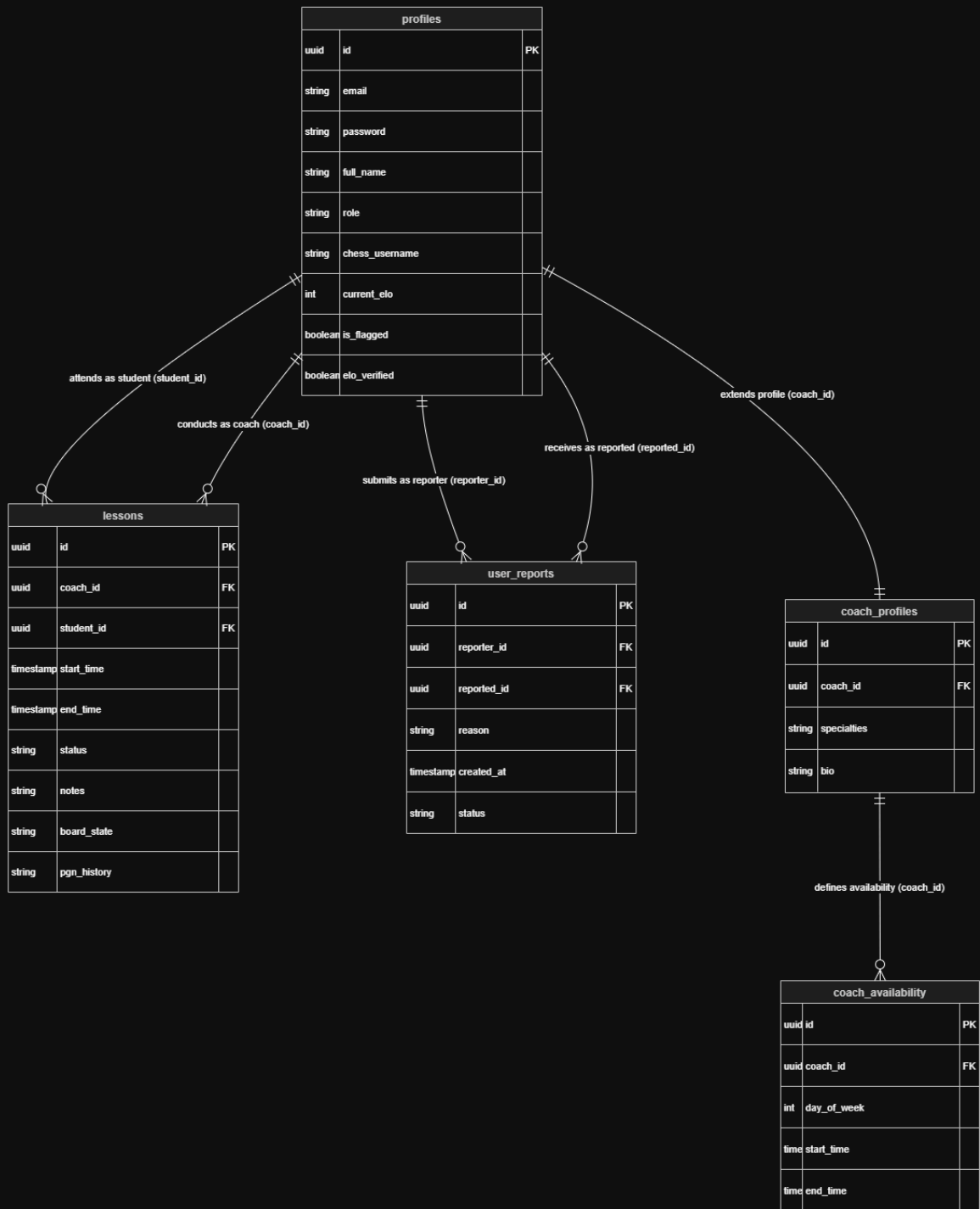
	<pre>{ "error": "Booking not available. This coach already has a session at this time." }</pre>
--	---

Common Error Codes

- **AUTH-001:** Invalid credentials
- **AUTH-002:** Token expired / Invalid Session
- **AUTH-003:** Insufficient permissions
- **COACH_UNAVAILABLE:** Time slot is outside the coach's configured schedule
- **TIME_CONFLICT:** Coach already has a lesson booked for this specific time
- **STUDENT_TIME_CONFLICT:** Student already has a conflicting lesson booked
- **SYSTEM-001:** Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-One:** profiles (Coach) ↔ coach_profiles (Each Coach user has exactly one extended public profile)
- **One-to-Many:** profiles (Coach) → coach_availability (A Coach can have multiple availability time slots)
- **One-to-Many:** profiles (Coach/Student) → lessons (Both Coaches and Students can participate in multiple lessons)
- **One-to-Many:** profiles (User) → user_reports (Users can submit or receive multiple conduct reports)

Key Tables:

1. **profiles** - Core user accounts, authentication credentials (hashed passwords), and base chess statistics.
2. **coach_profiles** - Extended public details (bio, teaching specialties) specifically for coach accounts.
3. **coach_availability** - Weekly schedule blocks defining a coach's open time slots.
4. **lessons** - Booking records, statuses, and live session data (FEN board state, PGN history, and Chat JSON).
5. **user_reports** - Trust & Safety logs submitted by users for Admin moderation.

Table Structure Summary:

- **profiles:** id (UUID), email, password (BCrypt), full_name, role, chess_username, current_elo, is_flagged, elo_verified
- **coach_profiles:** id (UUID), coach_id, specialties, bio
- **coach_availability:** id (UUID), coach_id, day_of_week, start_time, end_time
- **lessons:** id (UUID), coach_id, student_id, student_name, coach_name, start_time, end_time, status, notes, board_state, pgn_history
- **user_reports:** id (UUID), reporter_id, reported_id, reported_name, reason, created_at, status

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

<https://www.figma.com/design/AyBxlnYgMn7T5o9Vewb1KE/Untitled?node-id=0-1&t=XqGYmH0Oow5ak4Wk-1a>

Landing Page

Master the Board. Elevate Your Game.

Connect with expert coaches, track your ELO progress, and book real-time lessons — all in one elegant platform built for serious players.

Login

Login Page

CheckMateAcademy

Login Register

Welcome Future Grand Master

Username

Password

Login

Register Page

Register Page

CheckMateAcademy

Login Register

Student Coach

Username

Password

Repeat Password

Register

Register Page - Coach

CheckMateAcademy
Login
Register

Student
Coach

Username
Password
Repeat Password

ELO Rating
Coaching EXP
Achievements

Register

Player Pages

Player - Activity

CheckMateAcademy

Player

Login

Dashboard

Activity

Settings

Statistics

Player - Dashboard

CheckMateAcademy

Player

Login

Dashboard

Activity

Settings

Become a Coach

Player - Setting

CheckMateAcademy

Player

Login

Dashboard

Activity

Settings

Statistics

Player - Statistics

CheckMateAcademy

Player

Login

Player Dashboard

Active Statues

Active Statues

Active Statues

Active Statues

Active Statues

Active Statues

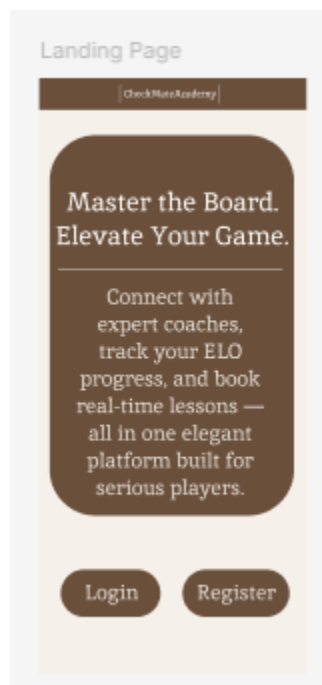
Coach Pages



7.2 Mobile Application Wireframes

<https://www.figma.com/design/AyBxINYgMn7T5o9Vewb1KE/Untitled?node-id=0-1&t=XqGYmH00ow5ak4Wk-1>

Landing Page



Login Page

Login Page

CheckMateAcademy

LOGIN

Username

Password

Login

Register Page

Register

CheckMateAcademy

Register

Student Coach

Username

Password

Repeat Password

Register

Register - Coach

CheckMateAcademy

Register

Student Coach

Username

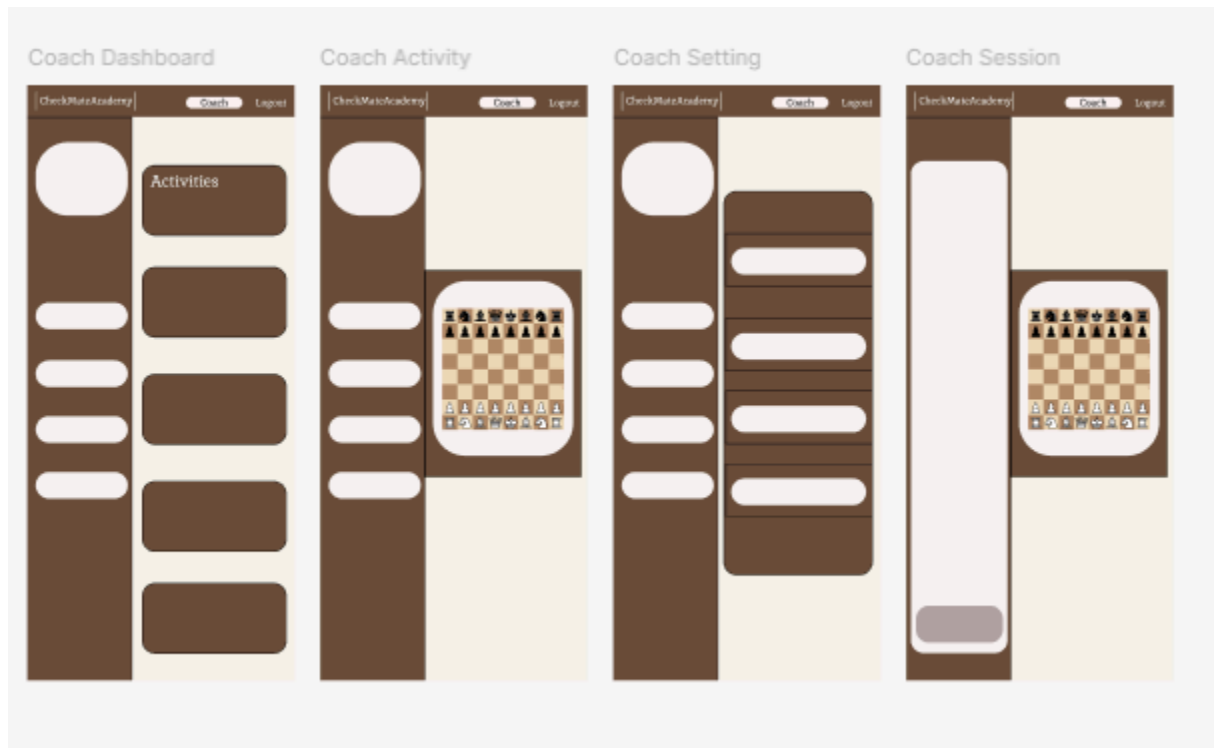
Password

Repeat Password

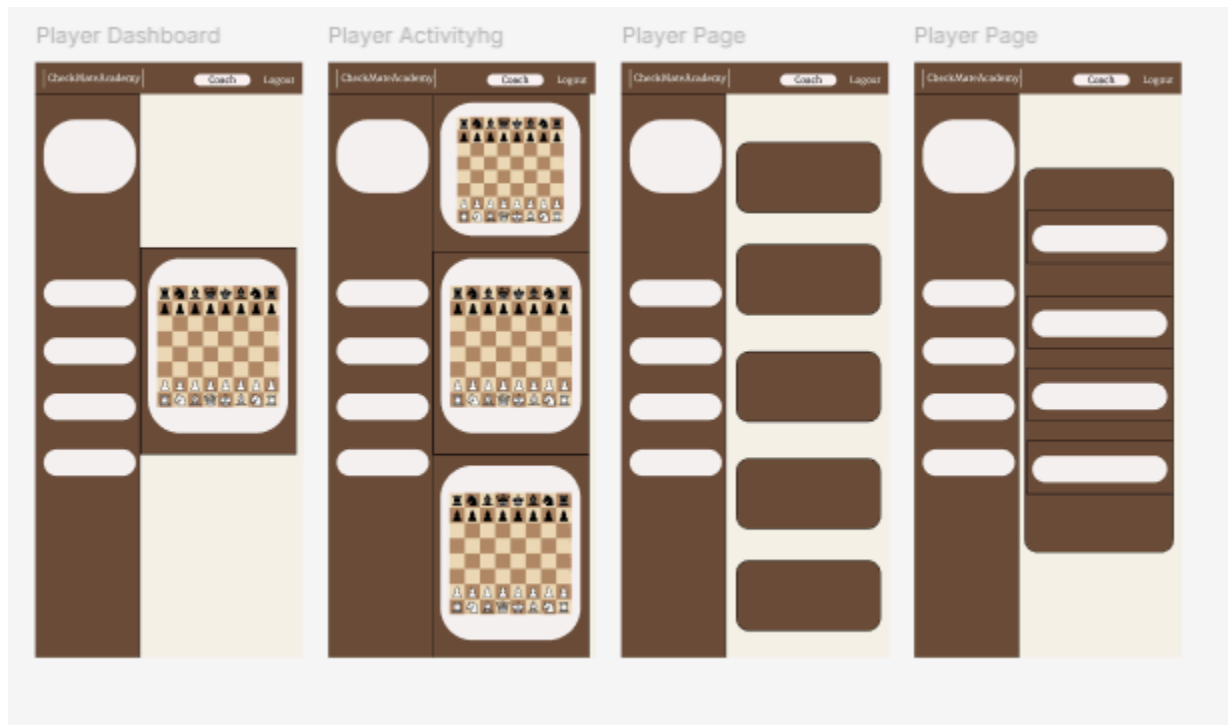
ELO Rating

Register

Coach Pages



Player Pages



Mobile-Specific Features:

- **Touch Optimization:** Touch-optimized buttons and selectable time slots (minimum 44x44px for accessibility).
- **Gesture Support:** Swipe to navigate between lesson days; pull-to-refresh on the dashboard for new booking requests.
- **Caching:** Offline caching for coach avatars and basic layout elements to speed up load times.
- **Responsive Navigation:** The desktop sidebar collapses into a bottom navigation bar or hamburger menu for mobile users to prioritize screen real estate for the chess board.
- **Input Simplification:** Simplified date/time picker forms for mobile lesson booking.

Design System:

- **Colors:** Primary (#6B4F3A), Secondary (#F5F1EA), Success (#10B981), Error (#EF4444)
- **Typography:** Arvo serif font family (Headings), Inter sans-serif font family (Body), responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

- Day 1-2: Project setup and documentation
- Day 3-4: Complete FRS and NFR
- Day 5-7: System architecture design

Week 2: Detailed Design

- Day 1-2: API specification
- Day 3-4: Database design
- Day 5-6: UI/UX wireframes
- Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

- Day 1: Spring Boot project setup and PostgreSQL configuration
- Day 2: Database schema and entity creation (profiles, availability)
- Day 3: Spring Security and BCrypt stateless auth implementation
- Day 4: Endpoint authorization and CORS policies setup
- Day 5: Coach directory endpoints and functions

Week 4: Core Features

- Day 1: Coach availability management
- Day 2: Lesson booking system
- Day 3: Search and filtering (by Elo rating/name)
- Day 4: Error handling and booking conflict validation
- Day 5: Admin Trust & Safety endpoints and API testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

- Day 1: React setup with Tailwind CSS
- Day 2: Authentication pages and sessionStorage logic (login, register)
- Day 3: Coach directory listing page
- Day 4: Coach profile and schedule view
- Day 5: Lesson booking calendar implementation

Week 6: Complete Web Features

- Day 1: Role-based routing (Student, Coach, Admin dashboards)
- Day 2: Lesson status management (Accept/Complete)
- Day 3: Live Session implementation (react-chessboard & 2000ms polling sync)
- Day 4: Responsive design polish
- Day 5: Spring Boot API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

- Day 1: Android Studio setup and project structure
- Day 2: Authentication screens
- Day 3: Coach browsing and search
- Day 4: Availability calendar view
- Day 5: Retrofit API integration (Spring Boot endpoints)

Week 8: Complete Mobile App

- Day 1: Lesson booking flow
- Day 2: User dashboards and lesson management
- Day 3: UI polish and animations
- Day 4: Testing on emulator/device
- Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

- Day 1: End-to-end testing across platforms
- Day 2: Bug fixes and optimization
- Day 3: Security review (validating RBAC and multi-tab isolation)
- Day 4: Performance testing
- Day 5: Documentation updates

Week 10: Deployment

- Day 1: Backend finalization (Render Web Service)
- Day 2: Web app deployment (Render Static Site with Rewrite rules)
- Day 3: Mobile APK distribution
- Day 4: Final testing
- Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)

2. Coach availability API (Week 3-4)
3. Lesson booking functionality (Week 4)
4. Dashboard and Live Session synchronization (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements